Software Requirements Specification

for

Voting System

Version 1.1 approved

Prepared by Ben Pullis (pulli068), Adam Douiri (douir002), Quang Pham (pham0327), Joe Anderson (and10287), Team 11

CSCI 4801

9-24-2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 3
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 4
3.3 Software Interfaces 4
3.4 Communications Interfaces 4
4. System Features 4
4.1 Receive File(s) 4
4.2 Seat Input 5
4.3 Algorithm Input 5
4.4 Determine # of Ballots and Candidates 6
4.5 Help Window 6
4.6 Election Details Display 7
4.7 Plurality Voting Algorithm 7
4.8 STV Voting Algorithm 8
4.9 Generate STV Audit Report 9
4.10 Shuffle Ballot Selection 9
5. Other Nonfunctional Requirements 10
5.1 Performance Requirements 10
5.2 Safety Requirements 10
5.3 Security Requirements 10
5.4 Software Quality Attributes 10
5.5 Business Rules 10
6. Other Requirements 10
Appendix A: Glossary 11
Appendix B: Analysis Models 11
Appendix C: To Be Determined List 11

Revision History

Name	Date	Reason For Changes	Version
SRS Draft	9-24-25	First draft	1.0
SRS Submission	10-1-25	Small fixes before submission	1.1

1. Introduction

1.1 Purpose

The purpose of this document is to display the detailed requirements of the voting software that handles the counting of ballots in an election run by an election official. This SRS aims to cover many areas of interest, including functionality, features, constraints, assumptions, dependencies, and more.

1.2 Document Conventions

This document was based on the IEEE System Requirement Specification Document template. We did not add any typographical conventions to the document to help prioritize certain information over other information.

1.3 Intended Audience and Reading Suggestions

Typical users and uses of this document include:

- An election official learning to use the system and its features for the first time
- Current programmers or engineers who need information to fix a bug
- Future programmers or engineers who are new to the codebase, looking to learn the system
- Stakeholders in the software who want a record of their software's requirements

This document is written with multiple parties in mind and contains a wide range of high and low-level technical details so all parties benefit from the document.

1.4 Product Scope

The voting system is an intricate piece of software that allows an election official the capabilities to smoothly and accurately determine the winner(s) of an election. The election official will input all the ballots, choose an election type, and enter the number of seats to be won. Once the system has all the information needed, it runs the counting process and delivers the results to the election official in the form of an election audit report.

1.5 References

IEEE Software Requirement Specification Document:

https://goo.gl/nsUFwy

Use Case Document:

Use case Document

Description From Product Owner:

Project Description

2. Overall Description

2.1 Product Perspective

This product is a part of a greater voting system in which there is voting and ballot files already generated. This system outlines a ballot counter (given existing ballot files) for simulating STV and plurality elections. It is broken down into three main components: a main interface, a help window, and an election results display.

2.2 Product Functions

There will be a main interface in which you can input the file names of the files containing ballots, enter the number of candidates, and select between plurality and STV voting algorithms. There will also be a help window, which can be pulled up at any point, that will give guidance to the user on how to set up an election. After the user input is complete, an election algorithm will run based on the provided input. Finally, there will be an election results display with the results gathered from the ballots and the election type.

2.3 User Classes and Characteristics

The users of this system will primarily be election officials. This user class will not have familiarity with command line arguments and should be able to be trained on the user interface within 15 minutes. They will be using the system about once per election.

2.4 Operating Environment

The operating environment of the software is within the terminal text-based user interface. The operating system this program will run on is Ubuntu, using the setup of the University of Minnesota CSE Labs machines.

2.5 Design and Implementation Constraints

STV Ballots must have at least 60% of the candidates ranked. For each Plurality ballot, each ballot will only have one candidate indicated. The user requires a report for each STV Election that acts

as an audit. Training for our user interface should take less than 15 minutes. An election should be able to be processed in under 5 minutes; there will be no more than 100,000 ballots for an election.

2.6 User Documentation

There will be a help menu provided within the user interface. The user can open the help window by typing ‘h’ into their command line at any point. This SRS document is also provided as software documentation.

2.7 Assumptions and Dependencies

We assume that the ballots have already been filed into a folder somewhere on the file system. We assume that the CSE Labs machine is correctly set up and does not have any existing issues or bugs with the operating system.

3. External Interface Requirements

3.1 User Interfaces

The voting system will provide a command-line interface to support election officials in conducting elections. The interface will be learnable within 15 minutes of training. The user interface includes the main interface, which opens on startup and prompts the user for input; the help window, which will provide the user with details about how to run the system; a results screen, which displays the results of the election; and an error screen, which displays an error message if the program fails.

1. Main Interface (Startup Screen)
    - Input prompt: File Name Input - Allows the user to specify the names of CSV files containing ballots.
    - Input prompt: Number of Seats - Allows the user to specify the number of seats to be filled for the election.
    - Input prompt: Voting Algorithm - Allows the user to choose between plurality or STV voting algorithms (“1” for STV or “2” for Plurality).
    - Input prompt: Help - Opens help window with instructions on how to operate the system (type “H”).
    - Input prompt: Disable Shuffle (STV Only) - Allows the user to disable the ballot shuffle for testing purposes.
    - Input prompt: Close Program - Terminates the program.
2. Help Window
    - Text information: Instructions - Step-by-step instructions on how to run the system.
    - Text information: Descriptions of Fields - Descriptions of all of the user input fields on the main interface.
    - Input prompt: Return to Main Interface - Returns the user to the main interface.
3. Election Details Display
    - Text information: Details - Display of election type, number of ballots processed, number of seats filled, and number of candidates.

- Text information: Results - Display results of the election, including elected and non-elected candidates, and either the percentage of votes received if the election type was plurality or the order in which candidates were elected/eliminated if STV.
- Input prompt: Return to Main Interface - Returns the user to the main interface.

4. Error Screen
    - Text information: Details - Descriptive message of the error message.
    - Input prompt: Return to Main Interface - Returns the user to the main interface.

3.2 Hardware Interfaces

The voting system requires a machine capable of running a CSE lab environment. The voting system requires standard input devices, such as a keyboard, in order for the user to provide text input. The voting system requires standard output devices, such as a screen, for the display of the text-based user interface.

3.3 Software Interfaces

1. Operating system: Ubuntu - Voting system uses the Ubuntu operating system used in the CSE lab machine environments. The system relies on file I/O operations for reading CSV files and writing audits. The system will also rely on standard I/O handling for the user interface.
2. Libraries and tools: C++ - System will utilize C++ (C++17 or newer) standard libraries for file I/O, user input handling, election data handling, and randomization when a tie occurs in the Plurality algorithm. The system will use Make for easy compilation of the program.
3. Data items: Input files - CSV files representing ballots will be input into the system. The first line will contain candidate names and their party, and the rest of the lines will contain rankings (STV) or votes (Plurality).
4. Data items: Output files - Audit report file. Text file with election metadata and results will be output from the system.

3.4 Communications Interfaces

The voting system does not require any external communications or networking capabilities. All data exchange occurs through local file I/O operations.

4. System Features

4.1 Receive file(s)

4.1.1 Description and Priority

Right when the program is started, the user is prompted to enter a filename containing ballots. The user can enter as many files of ballots as they would like, as long as they enter at least 1.

4.1.2 Stimulus/Response Sequences

Main Course:

1) The user starts the program
2) The user is prompted to enter the name of the file
3) The system stores the file in 1 location

Alternate Course:

1) The user starts the program
2) The user is prompted to enter the name of the file
3) The user has more files to be entered
4) The system prompts the user to add files until they are all entered
5) The system stores the files in 1 location

4.1.3 Functional Requirements

REQ-1: The user is prompted for input of the file name.
REQ-2: The system contains every single ballot that will be used to complete the election

4.2 Seat Input

4.2.1 Description and Priority

After starting the program, the election official will be prompted to enter the number of seats to fill for the election.

4.2.2 Stimulus/Response Sequences

1) The user starts the program
2) The user enters 1 or more files containing all ballots
3) The number of candidates is determined based on the ballots
4) The user is asked to provide a number of seats

4.2.3 Functional Requirements

REQ-1: The user is prompted for input of the number of seats.
REQ-2: The system stores an int number of seats, $0 < n < \#$ of candidates

4.3 Algorithm Input

4.3.1 Description and Priority

After starting the program, the election official will be prompted to enter the algorithm to use to decide the winners.

4.3.2 Stimulus/Response Sequences

1) The user starts the program
2) The user enters 1 or more files containing all ballots
3) The user enters the number of seats
4) The program asks the user to select an algorithm

4.3.3 Functional Requirements

REQ-1: The user is prompted for input of the election type.
REQ-2: The system stores the algorithm type as a boolean.

4.4 Determine # of ballots and candidates

4.4.1 Description and Priority

After the election official enters all of the ballots, the program must internally determine the # of ballots and # of candidates.

4.4.2 Stimulus/Response Sequences

1) The user has entered files of ballots and has declared finished entering files.
2) There is a variable, numCandidates, that can be determined based on ballot 1 by looking at how many options were on the ballot
3) There is a variable numBallots that starts at 0 and increments by 1 for each ballot
4) Go through each ballot in each file to determine the final ballot count

4.4.3 Functional Requirements

REQ-1: The user is prompted for the number of ballots and candidates.
REQ-2: The system stores ballots and candidates as variables
REQ-3: There are $0 < n$ ballots and $1 < n$ candidates.

4.5 Help Window

4.5.1 Description and Priority

When the user prompts the program for help, a help window should appear giving information about the program.

4.5.2 Stimulus/Response Sequences

1) The user prompts the program for help

2) The system presents a help window
3) The user can then prompt the program to return
4) The system will go back to the original window

4.5.3 Functional Requirements

REQ-1: The user can input ‘h’ at any point in the program, except for the running of an election.

REQ-2: The system will provide a help window to the user.

REQ-3: The user can exit out of this window back to their previous menu at any time.

4.6 Election Details Display

4.6.1 Description and Priority

After the election has been completed, the system will display a summary of the election results on the screen, including election details and the outcome for each candidate.

4.6.2 Stimulus/Response Sequences

Main Course:

1) Retrieve and display election type (STV or Plurality), number of ballots, number of seats, and number of candidates.
2) Display elected and non-elected candidates with the percentage of votes received, if the election type was Plurality.

STV Election Type Alternate Course:

1) Display the order of the winning and losing candidates for STV.

4.6.3 Functional Requirements

REQ-1: The system propagates election results and statistics to be displayed on screen.

4.7 Plurality Voting Algorithm

4.7.1 Description and Priority

After the user chooses the plurality algorithm in the algorithm input, a plurality algorithm will be used to determine the winner(s) of the election.

4.7.2 Stimulus/Response Sequences

Main Course:

1) A count is maintained for each candidate, starting at 0 for each
2) Each ballot is checked.
3) If a ballot chooses a candidate, the count for that candidate is increased by one.
4) The candidate(s) with the greatest number of votes will be declared the winner(s).

Alternate Course:
1) An electronic “fair coin toss” will determine the winner.

4.7.3 Functional Requirements

REQ-1: The system will read in the ballots.
REQ-2: The system will run the plurality voting algorithm on those ballots.
REQ-3: The system will store a winner.

4.8 STV Voting Algorithm (Droop Quota)

4.8.1 Description and Priority

After the user chooses the STV algorithm in the algorithm input, the Single Transferable Vote (STV) method using the Droop Quota will be applied to determine the elected and non-elected candidates.

4.8.2 Stimulus/Response Sequences

Main Course:

1) Shuffle ballots
2) Calculate Droop Quota
3) Distribute ballots into piles according to their first-choice candidate.
4) If any candidate reaches the quota, declare them elected and record their name. Their ballots are removed permanently. Subsequent ballots listing them will transfer to the next preferred candidate.
5) If no new candidate meets the quota, eliminate the candidate with the fewest votes. Redistribute their ballots according to the next preference.
6) Repeat steps 4–5 until all ballots are processed and all seats are assigned.
7) Create Elected and Non-Elected lists, ordered based on when they were elected and when they were placed on the non-elected list, respectively, and provide to the user

Alternate Course 1 (Tie occurs while eliminating a candidate):

1) The candidate who was last to receive their first ballot is eliminated
2) Return to Main Course step 5

Alternate Course 2 (User chooses to run test file):

1) Skip Main Course step 1
2) Return to Main Course step 2

4.8.3 Functional Requirements

REQ-1: The system will read in the ballots.
REQ-2: The system will run the STV voting algorithm on those ballots.
REQ-3: The system will store a winner.

4.9 Generate STV Audit Report

4.9.1 Description and Priority

At the end of the election, a .txt report will be automatically generated, including information on the type of election, the date, candidate names and parties, number of candidates, seats to be filled, elected list in order, and non-elected list in order.

4.9.2 Stimulus/Response Sequences

1) The election is run
2) The winner is declared, ending the election
3) The Report is generated in a .txt file with all the accurate information from the election
4) The report is placed somewhere that the user can access

4.9.3 Functional Requirements

REQ-1: The system will create an audit file as a .txt.

4.10 Shuffle Ballot Selection

4.10.1 Description and Priority

Before the STV election winner(s) can be determined, the election official decides if the ballots should be shuffled.

4.10.2 Stimulus/Response Sequences

1) The user chooses “1” (representing STV) in the algorithm input.
2) User is prompted whether they want ballots shuffled or not
3) If the user selects yes, the ballots are shuffled; otherwise, the ballots stay unshuffled

4.10.3 Functional Requirements

REQ-1: The user will be given a choice to shuffle the ballots in the STV algorithm.
REQ-2: If the user chooses not to shuffle the ballots, the ballots will remain unshuffled.
REQ-3: If the user chooses to shuffle the ballots, the ballots will be shuffled.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system processes any election, with up to 100,000 ballots, in under 5 minutes on a CSE lab machine. User training for the interface takes up to 15 minutes, at which point under 5 mistakes are made per hour of use.

5.2 Safety Requirements

The system ensures that no ballots are lost, duplicated, or modified throughout the entire election. All ballots are auditable and appear in the audit report. An electronic “fair coin toss” will be used for tie-breaking, which uses random number generation to ensure fairness. Similarly, the ballot shuffling process will also use secure randomization when shuffle is enabled.

5.3 Security Requirements

The system acknowledges that the security of the voting will be handled at the precinct itself, and the files received have been transmitted via a secure means externally. The system-generated audit reports for STV elections, including ballot movements and eliminations, will be provided for verification. Testing features (i.e., disabling shuffling) is restricted in final releases of the software.

5.4 Software Quality Attributes

The system offers a help window to assist users. The interface will be simple and easy to use by both experts and the average user. Users should have basic knowledge of elections and computers before using them.

5.5 Business Rules

Only the plurality and Droop quota algorithms are supported. STV ballots must have at least 60% of candidates ranked. Only authorized engineers and management have access to testing features.

6. Other Requirements

N/A

Appendix A: Glossary

Audit Report: A file that is automatically generated at the end of an election, containing key information from the election

Ballot Shuffle: Random shuffle of ballots before an STV election to ensure fairness

CSV File: A File where all values are separated by commas

Droop Quota: Number of votes a candidate must receive to be automatically elected in an STV election

Election Official: A Person running the program to conduct an election

Plurality Voting Algorithm: An algorithm where each ballot contains a single selection for whoever filled out the ballot wants to win

Seat: A spot that can be won by a candidate in the election

STV Voting Algorithm: An algorithm where voters rank the candidates from favorite to least favorite; at least 60% of candidates are ranked

User Interface: The way in which the user interacts with the program to receive prompts and give inputs

Appendix B: Analysis Models

N/A

Appendix C: To Be Determined List

N/A